

**EXERCISE 9.18**

Some information is lost when we combine parsers with the `or` combinator. If both parsers fail, we're only keeping the errors from the second parser. But we might want to show both error messages, or choose the error from whichever branch got *furthest* without failing. Change the representation of `ParseError` to keep track of errors that occurred in other branches of the parser.

9.7 Summary

In this chapter, we introduced *algebraic design*, an approach to writing combinator libraries, and we used it to design a parser library and to implement a JSON parser. Along the way, we discovered a number of combinators similar to what we saw in previous chapters, and these were again related by familiar laws. In part 3, we'll finally understand the nature of the connection between these libraries and learn how to abstract over their common structure.

This is the final chapter in part 2. We hope you've come away from these chapters with a basic sense of how functional design can proceed, and more importantly, we hope these chapters have motivated you to try your hand at designing your own functional libraries, for whatever domains interest *you*. Functional design isn't something reserved only for experts—it should be part of the day-to-day work done by functional programmers at all levels of experience. Before you start on part 3, we encourage you to venture beyond this book, write some more functional code, and design some of your own libraries. Have fun, enjoy struggling with design problems that come up, and see what you discover. When you come back, a universe of patterns and abstractions awaits in part 3.

